

POC Runbook: GCS Iceberg → Snowflake on AWS (Zero-Copy)

Companion to `iceberg-zero-copy-share-spec.md`. All commands in execution order. Syntax verified against Snowflake and Google Cloud docs as of 2026-07-13.

0. Prerequisites & conventions

- GCP project with billing enabled; you have `Owner` or equivalent for setup.
- Snowflake trial: sign up at signup.snowflake.com → **Enterprise** edition (default) → cloud **AWS** → region near your GCS region (e.g., GCS `us-central1` → Snowflake `AWS us-east-1` or `us-west-2`).
- Set a default warehouse once (`ALTER USER <username> SET DEFAULT_WAREHOUSE = COMPUTE_WH;`) — otherwise every new session must run `USE WAREHOUSE ...` before any `SELECT`.
- Note: GCP renamed BigLake → "Lakehouse for Apache Iceberg" (Apr 2026), but all APIs/CLI/IAM names still say `biglake`.

Set once (bash):

```
export PROJECT_ID="<your-gcp-project>"
export PROJECT_NUMBER="$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')"
export REGION="us-central1"
export BUCKET="iceberg-poc-${PROJECT_ID}"           # bucket name = catalog name (single-bucket mode)
export POOL_ID="snowflake-pool"
export PROVIDER_ID="snowflake-provider"
```

Phase 1 — GCP: catalog + storage

```
gcloud services enable biglake.googleapis.com --project $PROJECT_ID

gsutil mb -l $REGION -p $PROJECT_ID gs://$BUCKET

# Single-bucket catalog with credential vending (simplest; catalog name = bucket name).
gcloud biglake iceberg catalogs create $BUCKET \
  --project $PROJECT_ID \
  --catalog-type gcs-bucket \
  --credential-mode vended-credentials
```

Grant the catalog's auto-provisioned runtime service account storage access (required for vending; **not automatic**, and SA creation is eventually consistent — if the grant fails, wait a minute and retry):

```
gcloud biglake iceberg catalogs describe $BUCKET --project $PROJECT_ID # find the runtime SA email
gsutil iam ch serviceAccount:<runtime-sa-email>:roles/storage.objectUser gs://$BUCKET
```

Phase 2 — GCP: create a test Iceberg table

BigQuery **cannot** create tables in this catalog via DDL — use Spark. Cheapest path is a Dataproc Serverless batch (pennies). Save as `create_table.py`:

```
from pyspark.sql import SparkSession

CATALOG = "<BUCKET>"          # substitute literal values
PROJECT = "<PROJECT_ID>"

spark = (SparkSession.builder.appName("poc-create")
        .config(f"spark.sql.catalog.{CATALOG}", "org.apache.iceberg.spark.SparkCatalog")
        .config(f"spark.sql.catalog.{CATALOG}.type", "rest")
        .config(f"spark.sql.catalog.{CATALOG}.uri", "https://biglake.googleapis.com/iceberg/v1/restcatalog")
        .config(f"spark.sql.catalog.{CATALOG}.warehouse", f"gs://{CATALOG}")
        .config(f"spark.sql.catalog.{CATALOG}.header.x-goog-user-project", PROJECT)
        .config(f"spark.sql.catalog.{CATALOG}.header.X-Iceberg-Access-Delegation", "vended-credentials")
        .config(f"spark.sql.catalog.{CATALOG}.rest.auth.type", "org.apache.iceberg.gcp.auth.GoogleAuthManager")
        .config(f"spark.sql.catalog.{CATALOG}.io-impl", "org.apache.iceberg.gcp.gcs.GCSFileIO")
        .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
        .config("spark.sql.defaultCatalog", CATALOG)
        .getOrCreate())

spark.sql("CREATE NAMESPACE IF NOT EXISTS shared_aws")
spark.sql("""
CREATE TABLE IF NOT EXISTS shared_aws.orders (
  order_id BIGINT, customer STRING, amount DECIMAL(10,2), order_ts TIMESTAMP
) USING iceberg""")
spark.sql("""
INSERT INTO shared_aws.orders VALUES
(1,'acme',100.50,current_timestamp()),
(2,'globex',250.00,current_timestamp()),
(3,'initech',75.25,current_timestamp())""")
spark.sql("SELECT * FROM shared_aws.orders").show()
```

```
gsutil cp create_table.py gs://$BUCKET/jobs/
gcloud dataproc batches submit pyspark gs://$BUCKET/jobs/create_table.py \
--project $PROJECT_ID --region $REGION --version 2.3
```

Requires Iceberg 1.10+ runtime (Dataproc 2.3 serverless $\geq$ 2.3.10 includes `GoogleAuthManager`). Optional sanity check via BigQuery catalog federation: query the table from the BigQuery console to confirm the catalog is live.

Phase 3 — Snowflake ↔ GCP trust (workload identity federation)

3a. In Snowflake, get the issuer URL:

```
SELECT SYSTEM$GET_WORKLOAD_IDENTITY_ISSUER_URL();
```

3b. In GCP, create the pool + OIDC provider:

```

gcloud iam workload-identity-pools create $POOL_ID \
  --project $PROJECT_ID --location global

gcloud iam workload-identity-pools providers create-oidc $PROVIDER_ID \
  --project $PROJECT_ID --location global \
  --workload-identity-pool $POOL_ID \
  --issuer-uri "<ISSUER_URL_FROM_3a>" \
  --attribute-mapping "google.subject=assertion.sub"
# Audience: use the default audience.

```

Record the audience resource name:

```
//iam.googleapis.com/projects/$PROJECT_NUMBER/locations/global/workloadIdentityPools/$POOL_ID/providers/$PROVIDER_ID
```

3c. In Snowflake, create the catalog integration (as `ACCOUNTADMIN`):

```

CREATE OR REPLACE CATALOG INTEGRATION biglake_catalog_int
  CATALOG_SOURCE = ICEBERG_REST
  TABLE_FORMAT = ICEBERG
  REST_CONFIG = (
    CATALOG_URI = 'https://biglake.googleapis.com/iceberg/v1/restcatalog'
    CATALOG_NAME = 'gs://<BUCKET>'
    ADDITIONAL_HEADERS = ( "x-goog-user-project" = '<PROJECT_ID>' )
    ACCESS_DELEGATION_MODE = VENDED_CREDENTIALS
  )
  REST_AUTHENTICATION = (
    TYPE = OAUTH
    OAUTH_GRANT_TYPE = TOKEN_EXCHANGE
    OAUTH_TOKEN_URI = 'https://sts.googleapis.com/v1/token'
    OAUTH_AUDIENCE = '<AUDIENCE_RESOURCE_NAME_FROM_3b>'
    OAUTH_ALLOWED_SCOPES = ('https://www.googleapis.com/auth/bigquery')
  )
  ENABLED = TRUE;

DESC CATALOG INTEGRATION biglake_catalog_int;
-- Note WORKLOAD_IDENTITY_FEDERATION_SUBJECT from the output.

```

3d. In GCP, grant the Snowflake principal read access (least privilege — vending handles storage):

```

MEMBER="principal://iam.googleapis.com/projects/$PROJECT_NUMBER/locations/global/workloadIdentityPools/$POOL_ID/
subject/<WIF_SUBJECT_FROM_3c>"
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --role roles/biglake.viewer --condition=None --member "$MEMBER"
# The x-goog-user-project header makes Snowflake bill API usage to your project,
# which requires serviceusage.services.use:
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --role roles/serviceusage.serviceUsageConsumer --condition=None --member "$MEMBER"

```

Note: `CREATE OR REPLACE CATALOG INTEGRATION` mints a **new** WIF subject — rerun the grants (and remove the old subject's) after any replace.

3e. In **Snowflake**, verify before proceeding:

```
SELECT SYSTEM$VERIFY_CATALOG_INTEGRATION('biglake_catalog_int');
```

Phase 4 — Snowflake: catalog-linked database + query

```
CREATE OR REPLACE DATABASE shared_gcp_data
  LINKED_CATALOG = ( CATALOG = 'biglake_catalog_int' );

-- Discovery is async; give it a minute, then:
SHOW SCHEMAS IN DATABASE shared_gcp_data;      -- expect SHARED_AWS
SELECT * FROM shared_gcp_data.shared_aws.orders; -- 3 rows = zero-copy read works
```

If reads fail with a storage-access error, credential vending isn't flowing — fallback: create an external volume for `gs://<BUCKET>` (Snowflake generates a GCP service agent; grant it `roles/storage.objectViewer` on the bucket) and recreate the CLD with `EXTERNAL_VOLUME = <vol>`.

Phase 5 — Prove freshness (the zero-copy payoff)

Rerun a modified insert via Phase 2 (new `INSERT INTO shared_aws.orders VALUES (4,...)`), then immediately in Snowflake:

```
SELECT COUNT(*) FROM shared_gcp_data.shared_aws.orders; -- expect 4, no pipeline ran
```

Phase 6 — Measure egress

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.DATA_TRANSFER_HISTORY
WHERE transfer_type = 'DATA_LAKE'
ORDER BY start_time DESC; -- ACCOUNT_USAGE has up to ~2h latency
```

At POC scale this is cents; the point is confirming the meter works before production.

Teardown

```
DROP DATABASE shared_gcp_data;
DROP CATALOG INTEGRATION biglake_catalog_int;
```

```
gcloud biglake iceberg catalogs delete $BUCKET --project $PROJECT_ID
gsutil -m rm -r gs://$BUCKET
gcloud iam workload-identity-pools delete $POOL_ID --project $PROJECT_ID --location global
```

Troubleshooting

Symptom	Likely cause / fix
SYSTEM\$VERIFY_CATALOG_INTEGRATION auth error	Issuer URL or audience mismatch; recheck 3a/3b. Subject not yet granted (3d).
Verify fails: "Caller does not have required permission to use project"	WIF subject lacks roles/serviceusage.serviceUsageConsumer (needed for x-goog-user-project).
CLD create fails: "did not have credential vending enabled"	Recreate integration with ACCESS_DELEGATION_MODE = VENDED_CREDENTIALS in REST_CONFIG ; then re-grant the NEW WIF subject.
CLD created but no schemas appear	Discovery lag (wait), or biglake.viewer missing, or wrong CATALOG_NAME .
Table listed but SELECT fails on storage	Vending not honored → external-volume fallback (Phase 4). Also check runtime SA has storage.objectUser (Phase 1).
Spark job can't create table	Dataprocs version < 2.3.10 (no GoogleAuthManager) — pin --version 2.3 .
429s from catalog	BigLake API "Iceberg REST Catalog read requests per minute" quota — raise in Quotas & System Limits.
BigQuery can't see table	Expected for DDL; for querying use catalog federation, not five-part metadata-table names.

Known constraints for the POC: Parquet only; Iceberg V2/V3 tables only; no row/column-level security on catalog-managed tables (layer policies in Snowflake instead); metadata.json capped at 1 MB.

Sources

- [Snowflake: Configure a catalog integration for Google Cloud BigLake Metastore](#)
- [Snowflake: CREATE CATALOG INTEGRATION \(Iceberg REST\)](#)
- [Snowflake: Catalog-linked databases](#)
- [Google Cloud: Set up the Lakehouse Iceberg REST catalog endpoint](#)
- [Google Cloud: Configure workload identity federation](#)
- [Snowflake: Understanding data transfer cost](#)